VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi, Karnataka, INDIA

A Project Phase – II Report
on

Enhancing Prehospital Care Through AI and IoT

Submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering
in
Computer Science and Engineering

Submitted By

KRUTHI NL	**1GA22CS089**
MOHAN KUMAR M	**1GA22CS106**
PRABHU GOUDA	**1GA22CS119**
PUNEETH M	**1GA22CS123**

Under the Guidance of
Prof. Sowmya BK
Assistant Professor

SUSTAINABLE DEVELOPMENT GOALS OF THE PROJECT

SDG_NO: 3

Department of Computer Science and Engineering
GLOBAL ACADEMY OF TECHNOLOGY
Rajarajeshwarinagar, Bengaluru - 560 098

2025 – 2026

GLOBAL ACADEMY OF TECHNOLOGY

Department of Computer Science and Engineering

Accredited by NBA (2022-2025)

CERTIFICATE

Certified that the project work entitled **Enhancing Prehospital Care Through AI and IoT** carried out by Ms. **Kruthi NL,** USN **1GA22CS089,** Mr. **Mohan Kumar M,** USN **1GA22CS106,** Mr. **Prabhu Gouda,** USN **1GA22CS119,** Mr. **Puneeth M,** USN **1GA22CS123** bonafide student of **Global Academy of Technology** in partial fulfillment for the award of Bachelor of Engineering / Bachelor of Technology in **Computer Science** of the Visveswaraiah Technological University, Belgaum during the year 2025-26 It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of the Project work prescribed for the said Degree.

Signature of the Guide	Signature of the HoD	Signature of the Principal

GLOBAL ACADEMY OF TECHNOLOGY

Department of Computer Science and Engineering

Accredited by NBA (2022-2025)

DECLARATION

We, Kruthi NL, bearing USN 1GA22CS089, Mohan Kumar M, bearing USN 1GA22CS106, Prabhu Gouda, bearing USN 1GA22CS119, Puneeth M, bearing USN 1GA22CS123 students of Seventh Semester B.E, Department of Computer Science and Engineering, Global Academy of Technology, Raja Rajeshwari nagar Bengaluru, declare that the Project Work entitled "Enhancing Prehospital Care Through AI and IoT" has been carried out by us and submitted in partial fulfillment of the course requirements for the award of degree in Bachelor of Engineering in Computer Science and Engineering from Visvesvaraya Technological University, Belagavi during the academic year 2025 – 2026.

1.KRUTHI NL	**1GA22CS089**
2.MOHAN KUMAR M	**1GA22CS106**
3.PRABHU GOUDA	**1GA22CS119**
4.PUNEETH M	**1GA22CS123**

Place: Bengaluru

Date:

ABSTRACT

This project presents an advanced AI and IoT-enabled emergency response system designed to revolutionize pre-hospital care by significantly reducing critical delays in medical assistance. The system implements a comprehensive multi-stage pipeline that begins with intelligent accident detection using a hybrid Convolutional Neural Network (CNN) and Support Vector Machine (SVM) model, ensuring highly accurate incident identification with minimal false positives

For route optimization, we employ an advanced Deep Reinforcement Learning framework with Q- Networks that continuously learns from dynamic traffic patterns, road conditions, and historical response data to calculate the fastest possible route while accounting for real-time variables. The ambulance unit itself is equipped with a sophisticated array of IoT medical sensors that continuously monitor and record multiple patient vitals including heart rate, blood pressure, oxygen saturation, and ECG readings. These biometric data streams are processed through our specialized Long Short-Term Memory (LSTM) networks enhanced with Attention mechanisms, enabling precise real-time health assessment and early detection of critical conditions.

Our innovative hospital selection system implements the NSGA-II multi-objective optimization algorithm to evaluate and rank potential medical facilities based on a comprehensive set of criteria including: specialized treatment capabilities, current bed availability, estimated travel time, and the patient's specific medical requirements. This ensures optimal matching between patient needs and hospital resources. A secure, high-priority data channel transmits the patient's vital signs and preliminary diagnosis to the selected hospital in advance, allowing emergency staff to prepare necessary equipment, medication, and specialist teams before the ambulance arrives.

The system incorporates several novel features including adaptive learning mechanisms that improve performance over time, fail-safe protocols for network outages, and a user interface for emergency responders. By integrating these cutting-edge technologies, our solution addresses the most pressing challenges in emergency healthcare - including traffic congestion, inefficient resource allocation, and hospital unpreparedness - with the potential to improve survival rates by up to 40% during the critical golden hour.

ACKNOWLEDGEMENT

The satisfaction and the euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible. The constant guidance of these persons and encouragement provided, crowned our efforts with success and glory. Although it is not possible to thank all the members who helped for the completion of the Major Project Work (Phase – II) individually, we take this opportunity to express our gratitude to one and all.

We are grateful to the Management and our institute **GLOBAL ACADEMY OF TECHNOLOGY** with its very ideals and inspiration for having provided us with the facilities, which made this, Project Work a success.

We express our sincere gratitude to **Dr. Balakrishna H. B.,** Principal, Global Academy of Technology, for the support and encouragement.

We wish to place on record, our grateful thanks to **Dr. Kumaraswamy S**, Professor and Head, Department of CSE, Global Academy of Technology, for the constant encouragement provided to us.

We are indebted with a deep sense of gratitude for the constant inspiration, encouragement, timely guidance, and valid suggestions given to us by our guide **Prof Sowmya B K,** Department of CSE, Global Academy of Technology.

We are also thankful to the Project Coordinators, **Prof. Aparna K. S and Prof P. Sushmita Singh** and all the staff members of the Department for providing relevant information and helping in different capacities in carrying out Major Project Work.

KRUTHI NL 1GA22CS089

MOHAN KUMAR M 1GA22CS106

PRABHU GOUDA 1GA22CS119

PUNEETH M 1GA22CS123

CO-PO-PSO MAPPING AND JUSTIFICTAION OF THE PROJECT

Course: Major Project Phase II	**No. of Credits:06**
Course Code: CSE23706	**CIE Marks: 100, SEE marks:100**

Course Outcomes (CO)

Course out-comes	Descriptions
CO 1	Understand the requirements of the end user considering economic, social and environmental factors for providing a feasible solution with strong basics of Computer Science subjects.
CO 2	Design a feasible solution for the identified problem with good literature survey using modern tools.
CO 3	Prepare well-structured report of the project with plagiarism check and communicate the same in different phases / journals/ conferences / project exhibitions.
CO 4	Coordinate the assigned task with the team members within in specified time in concern with the project guide.

Program Outcomes (PO):

PO1: Engineering Knowledge **PO2**: Problem Analysis **PO3**:Design/Development of Solutions **PO4**: Conduct Investigations of Complex Problems **PO5**: Modern Tool Usage **PO6**: The Engineer and Society	**PO7**: Ethics **PO8**: Individual and Team Work **PO9**: Communication **PO10**: Project Management and Finance **PO11**: Life-long Learning

CO – PO Mapping:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	3				2	2				2	3	
CO2			3	3	3						2	3	
CO3								3			2	3	
CO4								3	3	3	2	3	

Program Specific Outcomes (PSO):

PSO1: Design, implement, and test system software and applications to meet real-world requirements.

PSO2: Develop effective solutions in communication networks, database systems, and computing applications

CO- PO-PSO Justification

CO1: ***Understand the requirements of the end user considering economic, social and environmental factors for providing a feasible solution with strong basics of Computer Science subjects.***

PO/PSO	Grade	Justification
PO1	3	CO1 requires applying core computer science knowledge to build feasible solutions.
PO2	3	Understanding user needs and constraints involves analyzing real-world problems and breaking them down into solvable components.
PO6	2	Considering social factors ensures the solution is ethical, inclusive, and relevant to societal needs.
PO7	2	CO1 emphasizes environmental awareness, encouraging eco-friendly and sustainable computing solutions.
PO11	2	CO1 require budgeting, resource planning, and cost-effective implementation strategies.
PSO1	3	CO1 directly aligns with PSO1 by using foundational computer science principles to design practical, user-centric solutions.

CO2: ***Design a feasible solution for the identified problem with good literature survey using modern tools.***

PO/PSO	Grade	Justification
PO3	3	CO2 emphasizes designing a practical solution based on user requirements and literature survey. This directly aligns with PO3, which focuses on developing system components or processes that meet specified needs.
PO4	3	Literature survey and feasibility analysis involve investigating existing research, methodologies, and technologies. This matches PO4, which requires using research-based knowledge and methods to arrive at valid conclusions.
PO5	3	CO2 requires applying modern tools to implement and validate the solution. This aligns with PO5, which stresses the use of appropriate techniques and IT tools in engineering practice.
PO11	2	CO2 require budgeting, resource planning, and cost-effective implementation strategies.
PSO1	3	CO2 directly aligns with PSO1 by using foundational computer science principles to design practical, user-centric solutions.

CO3: ***Prepare well-structured report of the project with plagiarism check and communicate the same in different phases / journals/ conferences / project exhibitions.***

PO/PSO	Grade	Justification
PO8	3	Preparing a plagiarism-free, well-structured report and sharing it in journals or conferences reflects academic honesty, respect for intellectual property, and professional responsibility. It ensures integrity in communication and compliance with ethical norms of engineering practice.
PO11	2	CO3 require budgeting, resource planning, and cost-effective implementation strategies.
PSO1	3	CO3 directly aligns with PSO1 by using foundational computer science principles to design practical, user-centric solutions.

CO4: ***Coordinate and execute the assigned task and evaluate with the team members within in specified time in concern with the project guide.***

PO/PSO	Grade	Justification
PO8	3	Preparing a plagiarism-free, well-structured report and sharing it in journals or

		conferences reflects academic honesty, respect for intellectual property, and professional responsibility. It ensures integrity in communication and compliance with ethical norms of engineering practice.
PO9	3	CO4 emphasizes working effectively in a team, coordinating tasks, and collaborating with peers under the guidance of the project mentor. This directly aligns with PO9, which focuses on functioning as an individual and as a member/leader in diverse teams.
PO10	3	CO4 requires clear communication with team members and the project guide to execute tasks, share progress, and evaluate outcomes. This aligns with PO10, which stresses effective communication in reports, presentations, and discussions.
PO11	2	CO4 require budgeting, resource planning, and cost-effective implementation strategies.
PSO1	3	CO4 directly aligns with PSO1 by using foundational computer science principles to design practical, user-centric solutions.

TABLE OF CONTENTS

Chapter No.	Particulars	Page. No
	Abstract	i
	Acknowledgment	ii
	CO-PO-PSO Mapping	iii
	Table of Contents	v
	List of Figures	vi
	List of Tables	vii
	Glossary	viii
1	Introduction	1
	1.1 Introduction	1
	1.2 Problem Definitions	2
	1.3 Existing Systems	3
	1.4 Proposed System	4
	1.5 Objectives of the Project Work	4
	1.6 Scope of the Project Work	5
	1.7 Project Report Outline	6
2	Literature Survey	4
	2.1 System Study	7
	2.2 Review of Literature	8
	2.3 Comparison of Literature	9
3	System Requirement Specification	11
	3.1 Functional Requirements	11
	3.2 Non-Functional Requirements	12

	3.3 Hardware Requirements	13
	3.4 Software Requirements	14
4	System Design	16
	4.1 Design Overview	16
	4.2 System Architecture	17
	4.3 Data Flow Diagrams	18
	4.3.1 DFD- Level 0	18
	4.3.2 DFD- Level 1	18
	4.3.3 DFD- Level 2	19
	4.4 Use Case Diagram	19
	4.5 Class Diagram	20
	4.6 Sequence Diagram	21
	4.7 Collaboration Diagram	22
	4.8 Activity Diagram	23
	4.9 Modules	24
	4.9.1 Ambulance Dispatch Module	24
	4.9.2 Dynamic Route Optimization Module	24
	4.9.3 IoT-Based Patient Monitoring Module	24
5	Implementation	25
	5.1 Steps for Implementation	25
	5.2 Implementation Issues	26
	5.3 Algorithms	27
	5.3.1 Ambulance Routing Algorithm	27
	5.3.2 Patient Monitoring Algorithm	27
	5.3.3 Hospital Recommendation Algorithm	27
6	Testing	28
	6.1 Test Environment	28
	6.2 Individual Module Unit Testing	29
	6.3 Integration Testing of Modules	30
	6.4 System Testing	30
	6.5 Functional Testing	31
7	Results	32

8	Conclusions	37
	8.1 Major contribution	38
	8.2 Future Enhancements	39
	Bibliography	40

LIST OF FIGURES

Figure No.	Figure Name	Page. No
Figure 4.1	System Architecture	13
Figure 4.2	Data Flow Diagram - Level 0	14
Figure 4.3	Data Flow Diagram - Level 1	14
Figure 4.4	Data Flow Diagram - Level 2	15
Figure 4.5	Use Class Diagram	16
Figure 4.6	Class Diagram	17
Figure 4.7	Sequence Diagram	18
Figure 4.8	Collaboration Diagram	20
Figure 4.9	Activity Diagram	21

List of Tables

Table No.	Figure Name	Page. No
Table 2.3	Comparison of Literature	11
Table 6.2.1	User Login and Registration	13
Table 6.2.2	Uploading Video/image/audio	14
Table 6.2.3	Frame Extraction & Audio Processing	14
Table 6.2.4	Model Inference	15
Table 6.2.5	Blockchain Hash Verification	16

GLOSSARY

Term	Meaning
ISL	Indian Sign Language
DFD	Convolutional Neural Network
Gesture Recognition	Identification of hand signs using ML
Emotion Detection	Identifying emotions from facial expressions
ROI	Region of Interest
Dataset	Collection of images used for model training
DHH	Deaf and Hard-of-Hearing
LSTM	Long Short-Term Memory
TTS	Text-to-Speech

CHAPTER 1

Introduction

1.1 Introduction

In rapidly urbanizing regions like Bangalore, emergency medical services face critical challenges that significantly impact patient survival rates. The current system suffers from inefficient ambulance dispatch procedures, lack of real-time patient data during transit, traffic congestion delays, and suboptimal hospital selection - issues that are further exacerbated in rural areas with limited healthcare infrastructure. These shortcomings often result in missed golden hours, the crucial first 60 minutes after trauma when prompt medical intervention is most effective.

Our project addresses these systemic gaps through an integrated AI and IoT solution that revolutionizes the entire emergency response chain. The system begins with intelligent accident detection using computer vision algorithms, followed by optimized ambulance dispatch and dynamic route planning. Inside the ambulance, IoT sensors continuously monitor and transmit patient vitals, while our AI engine analyzes this data to select the most appropriate hospital based on multiple critical parameters. The solution also incorporates smart traffic management to prioritize emergency vehicles.

This technological intervention aims to transform emergency healthcare delivery by:

1. Reducing response times through automated dispatch and optimized routing
2. Improving treatment outcomes via real-time vital monitoring
3. Ensuring optimal hospital selection based on current capacity and specialization
4. Minimizing transit delays through intelligent traffic coordination

By bridging these critical gaps in prehospital care, our system has the potential to significantly improve survival rates and set new standards for emergency medical services in both urban and rural environments. The integration of cutting-edge AI algorithms with robust IoT infrastructure creates a scalable solution that can be adapted to various healthcare ecosystems while maintaining cost-effectiveness and reliability.

1.2 Problem Definitions

SVM (Support Vector Machine): A supervised learning algorithm that classifies data by finding the optimal decision boundary between categories, maximizing the margin between classes. It excels in high-dimensional spaces and is effective for binary classification tasks like accident detection.

CNN (Convolutional Neural Network): A deep learning model designed for processing grid-like data (e.g., images), using convolutional layers to automatically detect hierarchical patterns. It enables accurate accident identification by analysing visual inputs from cameras or sensors.

NSGA-II (Non-dominated Sorting Genetic Algorithm II): An evolutionary optimization method for multi-objective problems, sorting solutions into Pareto-optimal fronts while maintaining diversity. It selects the best hospital by balancing factors like distance, specialization, and resource availability.

Deep Q-Networks (DQN): Combines Q-learning with deep neural networks to handle complex decision-making in dynamic environments. It optimizes ambulance routes by learning from traffic data and rewards efficient paths.

LSTM (Long Short-Term Memory): A recurrent neural network variant with memory cells to process sequential data (e.g., patient vitals over time). Its attention mechanism prioritizes critical health trends for real-time analysis.

IoT (Internet of Things): A network of interconnected physical devices embedded with sensors, software, and connectivity to collect and exchange data over the internet. It enables smart, automated systems by bridging the physical and digital worlds.

IoT Sensors: Specialized components that detect and measure real-world conditions (e.g., heart rate, location, temperature) and convert them into digital data. They form the backbone of IoT systems by providing real-time inputs for analysis and decision-making.

1.3 Existing System

The current emergency medical response system still relies heavily on manual processes and traditional communication methods. In most cases, accidents are reported by people at the scene or through police phone calls. Because these reports depend on human involvement, many incidents are either detected late or not reported immediately. This delay directly affects how quickly an ambulance can reach the patient.

Ambulance dispatching also follows a manual approach, where operators decide which ambulance should respond based on general information. There is no intelligent system to automatically suggest the nearest or fastest option. As a result, response times are often longer than necessary, especially during peak traffic hours or in busy city areas.

Another major limitation in the existing system is **routing**. Ambulances usually depend on normal maps that do not consider real-time road conditions, traffic jams, or roadblocks. Since the routes are not optimized dynamically, ambulances can get stuck in traffic, causing further delays in reaching the patient and the hospital.

During transportation, patient monitoring is basic and sometimes completely absent. Vitals such as heart rate, oxygen level, or blood pressure are either checked manually or not recorded continuously. Since hospitals do not receive any real-time updates, medical teams start their preparation only after the patient arrives, losing valuable time during emergencies.

Hospital selection is also not data-driven. Ambulances are often sent to the nearest hospital without knowing if beds, doctors, or required facilities are available. This leads to situations where the ambulance has to be redirected to another hospital, causing additional delays during critical moments.

Overall, the existing system works, but it is slow, disconnected, and dependent on human judgement at many stages. There is very little automation, no real-time intelligence, and no continuous communication between the ambulance, hospital, and control room. These limitations highlight the need for a modern, unified system that uses AI and IoT to improve speed, accuracy, and patient outcomes. .

1.4 Proposed System

The proposed system introduces an intelligent, fully automated emergency response framework that integrates AI, IoT, and optimization algorithms to drastically improve prehospital care. Unlike traditional manual processes, this system begins by automatically detecting accidents using a hybrid CNN–SVM model, ensuring faster and more accurate identification of emergency events. Once an incident is detected, the system immediately locates and dispatches the nearest available ambulance using real-time GPS and availability data.

To minimize travel time, the ambulance route is optimized through a Deep Reinforcement Learning model that continuously learns from traffic behavior, congestion, and road conditions. Inside the ambulance, IoT-enabled medical sensors constantly monitor patient vitals such as heart rate, SpO_2, temperature, and ECG patterns. These readings are sent to the hospital in real time so that medical teams can prepare before the patient arrives.

A multi-objective decision-making algorithm (NSGA-II) then evaluates nearby hospitals by considering specialization, bed availability, distance, and the patient's condition. The system selects the best possible hospital and communicates the ETA and vitals to the medical team. A complete emergency report—containing event details, dispatch time, route logs, and vitals—is generated automatically.

Overall, the proposed system delivers a fast, smart, and reliable emergency response pipeline that bridges the gaps in existing EMS infrastructure and improves patient survival during the golden hour.

1.5 Objectives of the Project Work

The main objective of this project is to design a unified, AI-driven prehospital care system that automates accident detection, ambulance dispatching, patient monitoring, and hospital selection. The system aims to reduce emergency response time and ensure that the patient receives medical attention as quickly as possible.

A key goal is to use a hybrid CNN + SVM model for accurate accident recognition and implement Deep Reinforcement Learning to create the fastest possible route for the ambulance. Another major objective is to enable continuous patient monitoring using IoT sensors and send these vitals to the hospital in real time, ensuring immediate medical preparedness.

The project also aims to use NSGA-II for providing the best hospital recommendation based on multiple factors such as specialization, bed availability, and distance. Along with this, the

system seeks to generate structured emergency reports, maintain a secure database of incidents, and provide a user-friendly interface for operators and healthcare providers.

Ultimately, the objective is to create a scalable, efficient, and real-world-ready emergency response system that enhances the quality of prehospital care and saves lives through technology-driven decision making.

1.6 Scope of the Project Work

The scope of this project covers the entire cycle of emergency medical response, from accident detection to hospital admission. It includes route optimization through DRL, continuous patient vitals monitoring with IoT sensors, and intelligent hospital recommendation using multi-objective optimization.

Functionally, the system offers real-time data synchronization between ambulance and hospital, a dashboard for monitoring emergencies, and automated emergency report generation. The database stores accident records, patient vitals, route history, and hospital details for later analysis.

The project is highly relevant to urban and rural EMS, traffic management authorities, smart city initiatives, hospitals, and emergency operators. In future, the system can be expanded by adding features such as real-time traffic signal control for ambulances, automated drone-based first response, integration with national health records, and large-scale deployment across multiple districts or states.

Overall, the scope ensures that the system is scalable, practical, and capable of handling real-world emergency scenarios effectively.

1.7 Project Report Outline

This report is organized into well-structured chapters that cover the motivation, design, implementation, and evaluation of the AI- and IoT-enabled emergency response system.

Chapter 1 – Introduction

Provides an overview of emergency medical challenges, explains the need for an AI-integrated solution, and outlines the project's objectives and scope.

Chapter 2 – Review of Literature

Summarizes previous studies on accident detection, ambulance routing, IoT-based monitoring, and hospital triage. Highlights the limitations of existing systems and the need for an integrated approach.

Chapter 3 – System Requirement Specification

Details both functional and non-functional requirements, and lists the hardware, software, tools, and technologies required to build the system.

Chapter 4 – System Design

Explains the architecture of the proposed system along with DFDs, UML diagrams, module descriptions, and the workflow of the entire emergency response pipeline.

Chapter 5 – Implementation

Describes how AI models, IoT devices, route optimization, and hospital selection algorithms are implemented. Covers backend logic, interfaces, and integration steps.

Chapter 6 – Testing

Provides test cases, scenarios, and results validating the performance, reliability, and accuracy of the system.

Chapter 7 – Results and Discussions

Presents screenshots, visual outputs, performance graphs, and a discussion on how well the system meets its goals.

Chapter 8 – Conclusions

Summarizes the achievements of the project and explains its potential impact on EMS services.

Chapter 9 – Bibliography

Lists all research papers, references, and sources used.

Chapter 10 – Publication/Communications

Includes any publications, presentations, or conference submissions related to the project.

CHAPTER 2

Review of Literature

The summary of the paper titled "Ambulance Route Optimization in a Mobile Ambulance Dispatch System Using Deep Neural Network (DNN)" is provided below: The evolution of emergency medical response has seen significant improvements through the adoption of GPS- based tracking, traffic-aware routing algorithms, and IoT-enabled health monitoring. Traditional systems, while effective to a degree, often function in silos—delaying optimal decision-making during the golden hour of trauma care. Previous research has employed CNNs and SVMs for accident detection, and deep reinforcement learning for traffic navigation, yet these approaches lack real-time coordination across the full emergency chain. Additionally, hospital triage and selection have typically relied on static or manual methods, limiting adaptability in critical moments. These technological advances inform the foundation of our work, while also highlighting the need for a fully integrated, intelligent, and dynamic emergency dispatch system.

2.1 System Study

Research into intelligent emergency medical services has explored a range of technologies to improve response times, route optimization, and patient monitoring. The most prominent approaches include:

- **Accident Detection Using Deep Learning:** Convolutional Neural Networks (CNN) and Support Vector Machines (SVM) have been widely used for real-time accident detection from CCTV or dashboard camera feeds. These models have demonstrated effective classification of traffic scenarios but are rarely integrated into full dispatch systems.
- **Ambulance Dispatch and Dynamic Routing:** Several works apply traditional Dijkstra or A* algorithms for ambulance routing. More recent studies explore Deep Reinforcement Learning (DRL), including Q-learning and DQN, to adapt routes based on real-time traffic and road conditions. However, many lack hospital-level triage or resource-awareness.
- **Patient Monitoring via IoT Sensors:** IoT-based wearable and in-vehicle sensors are increasingly used to capture vital signs like heart rate, blood pressure, and oxygen saturation during transit. While effective for monitoring, these systems often operate independently of the dispatch network and hospital backend.
- **Hospital Recommendation Systems:** Efforts to automate hospital selection often use static distance-based logic. Recent work explores multi-objective optimization algorithms like NSGA-II for better decisions based on hospital capacity, specialization, and proximity, though

they are rarely linked with real-time vitals or DRL systems.

- **Integrated AI-IoT Frameworks (Gap Identified):** While each of these systems has shown promise in its specific domain—accident detection, routing, patient monitoring, or triage—few studies integrate them into a unified AI-driven, IoT-enabled emergency response pipeline. This highlights a significant gap in the literature.

  Our project addresses this by combining automated accident detection (CNN+SVM), dynamic dispatch and route optimization (DRL), live patient monitoring (IoT), and intelligent hospital selection (NSGA-II) into a single decision-support framework for next-generation emergency medical response.

2.2 Review of Literature

Researchers have explored a variety of AI-based EMS enhancements. Early works primarily focused on single-task implementations. For example, computer vision models like CNNs and YOLO variants were widely used for detecting road incidents and emergency vehicles. While these models performed well on curated datasets, they often struggled with real-world environments involving occlusion, poor lighting, or dense traffic.

Deep reinforcement learning models such as DQN and A3C have shown promise in optimizing ambulance routes by learning traffic patterns. However, many of these approaches did not consider real-time hospital availability or patient severity, limiting their effectiveness in end-to-end decision-making.

IoT systems are increasingly used in ambulances to monitor vitals. Studies using wearable sensors, ECG modules, and cloud-based dashboards demonstrated that continuous monitoring can significantly improve hospital preparedness. However, most systems lacked adaptive intelligence for triage and did not support predictive analysis during transit.

On the hospital selection side, NSGA-II and other evolutionary algorithms have been used for multi-objective decision optimization. These methods consider factors like distance, medical facilities, and availability but often lack integration with live patient data collected during transportation.

Integration-based works exist but remain limited. Some focus on traffic priority systems, enabling emergency vehicles to control traffic lights, while others experiment with cloud-assisted emergency routing. Yet, very few systems jointly use DRL for routing, IoT sensors for vitals, NSGA-II for hospital triage, and CV-based accident detection.

The proposed system addresses this gap by integrating all major EMS components into a unified, intelligent, and autonomous emergency response platform.

2.3 Comparison of Literature

Author/ Year	Technique Used	Modulity	Limitation Identified	Relevance to Proposed System
Selvan et al., 2025	Deep Neural Networks for Routing	Ambulance Routing	Does not consider hospital readiness	Forms baseline for DRL-based route optimization
Noor et al., 2024	YOLOv8 for Emergency Vehicle Tracking	Traffic / Visual Detection	Limited to traffic recognition	Highlights need for multimodal fusion
Pagare et al., 2024	AI Traffic Control	Traffic Prioritization	Limited to traffic signals only	Supports future integration of traffic light automation
Batool, 2025	5G IoT Monitoring	Patient Monitoring	Tracks vitals but lacks triage	Encourages IoT-based monitoring in ambulance
Sakthidevi et al., 2024	IoT Smart Ambulance	IoT / EMS	No hospital selection system	Helps define IoT architecture and vitals flow
Bajwa, 2025	AI EMS Review	EMS Systems	Does not integrate components	Confirms need for unified emergency response
Al-Shareeda et al., 2025	Digital Twin Tracking	Emergency Routing	Limited real-time application	Supports location-based hospital recommendation

Rautenstrauß et al., 2025	Optimi-zation-ML	EMS Operations	Lacks live patient data	Encourages ML + optimi-zation use
Pagare et al., 2024	YOLO Traffic Priority	Traffic Signals	Works in limited setups	Supports traf-fic-based routing im-provement
Sakthidevi et al., 2024	IoT Am-bulance	IoT	Not scalable for large networks	Guides scala-ble IoT mod-ule design

CHAPTER 3

System Requirement Specification

This chapter outlines the various system requirements necessary for the successful development and deployment of the intelligent emergency medical response system. The requirements are categorized into functional, non-functional, hardware, and software needs. These specifications ensure that the system performs reliably, securely, and efficiently in time-critical situations such as accident detection, ambulance dispatch, and patient triage.

3.1 Functional Requirements

Ambulance Dispatch Automation

The system shall track the real-time locations of available ambulances. It shall dispatch the nearest ambulance to the accident site upon verification. It shall notify both the traffic control system and receiving hospital.

Dynamic Route Optimization

The system shall calculate the optimal route using Deep Q-Learning based on live traffic conditions. It shall continuously adapt the route in case of roadblocks or congestion. Navigation instructions shall be updated in real time.

Patient Monitoring via IoT Sensors

The ambulance shall collect patient vitals (heart rate, oxygen saturation, temperature) using IoT devices. Data shall be securely transmitted to the receiving hospital during transit. Alerts shall be raised for any abnormal vital readings.

Hospital Selection Using NSGA-II

The system shall assess hospitals based on distance, specialization, capacity, and live patient condition. It shall recommend the most suitable hospital using a multi-objective optimization strategy. Final hospital selection and ETA shall be shared with the medical team.

Emergency Report Generation

An auto-generated summary report will be created that consolidates all critical information, including incident details, the ambulance routing log, patient vital statistics recorded during transit, and the final hospital triage decision.

3.2 Non-Functional Requirements

Performance

Response time from detection to dispatch shall be reduced. Route recalculation shall occur for change in traffic condition. Vitals monitoring shall refresh at certain interval of time.

Scalability

System shall support at least 50 concurrent emergency cases. It shall be extensible to nationwide deployment with 1000+ ambulance units.

Reliability

Mission-critical services shall maintain uptime of 99.99%. The system shall cache fallback routing paths and allow offline dispatch if needed.

Usability

Dashboard shall offer intuitive controls and map-based visuals for operators. Mobile/tablet compatibility shall be ensured for field operatives.

Security

All sensor and patient data shall be secured. Role-based authentication shall control access to different modules.

Compliance

Emergency logs and reports shall follow the National Health Authority (NHA) and Indian Medical Emergency standards. Audit logs and access tracking shall be available for legal and clinical accountability.

3.3 Hardware Requirements

To develop and run the AI-based emergency response system efficiently, we need hardware that can smoothly handle tasks like accident detection, IoT data processing, routing, video analysis, and database management. The following hardware setup ensures reliable performance during both development and deployment:

• Processor (CPU)

A modern, high-speed processor such as Intel Core i5/i7 or AMD Ryzen 5/7 is preferred. These CPUs are capable of handling machine learning computations, video processing, and backend operations without lag.

• RAM (Memory)

At least 8 GB of RAM is required, but 16 GB is recommended to smoothly manage multitasking, real-time data processing, and running multiple services such as the web server, routing engine, and IoT modules simultaneously.

• Storage

A Solid State Drive (SSD) is highly recommended. It speeds up program execution, database access, and model loading. Around 256 GB or more is ideal for storing logs, datasets, and system files.

• Operating System

The system can run on Windows 10/11, Ubuntu Linux, or macOS. However, Ubuntu Linux provides the best compatibility with open-source AI tools, Python libraries, and backend servers used for this project.

• Internet Connectivity

A stable internet connection is needed for installing dependencies, fetching map APIs, real-time cloud sync, and updating the system. Major features like accident detection and routing can work offline once set up, but modules like live traffic updates require internet access.

• Display and Peripherals

1080p monitor, keyboard, and mouse are necessary for development, visualization of maps, and analysis of accident detection results. Better resolution helps when working with image and video-based models.

3.4 Software Requirements

The emergency response system combines AI, IoT, machine learning, web technologies, and database tools. The system requires several software components to work together efficiently.

• Programming Language: Python 3.x

Python serves as the core programming language due to its simplicity and extensive support for AI, ML, and data processing libraries.

• Development Tools / IDEs

Tools like VS Code, PyCharm, Jupyter Notebook, or Anaconda are recommended for writing, debugging, and testing the code. These IDEs make it easier to handle multiple modules such as accident detection, routing scripts, IoT signal processing, and server-side programming.

• AI and Machine Learning Libraries

TensorFlow / Keras is used for training and deploying models like CNNs for accident detection, classification, and image recognition. Scikit-learn is helpful for preprocessing, model evaluation, clustering, and supporting ML algorithms. OpenCV is essential for image and video processing, frame extraction, accident detection, and vehicle tracking.

• Supporting Python Libraries

NumPy is used for high-speed mathematical operations and numerical calculations. Pandas is used for maintaining logs, working with datasets, and handling structured data. SciPy is useful for statistical analysis and additional signal processing. Matplotlib and Seaborn – Required for visualizing training curves, detections, routes, and sensor graphs.

• Web Technologies

Flask serves as the backend framework for user authentication, file uploads, IoT data handling, and connecting AI outputs to the frontend. HTML5, CSS3, Bootstrap, JavaScript is used to build a simple and responsive dashboard for ambulance tracking, detection results, and hospital suggestions.

• Database

MongoDB stores user accounts, ambulance records, accident logs, IoT vitals, and recommended hospital details. It offers a clean graphical interface and supports fast queries for real-time emergency workflows.

• Mapping & Routing Support

Integration with Google Maps API / OpenStreetMap for calculating shortest paths, traffic-based routing, and ambulance navigation.

CHAPTER 4

System Design

This chapter describes the design aspects of the AI-IoT-Based Emergency Medical Response System in detail, including architectural layout, module-level breakdown, and system interaction flow. The design ensures high responsiveness, modularity, and real-time intelligence across detection, dispatch, routing, monitoring, and hospital coordination.

4.1 Design Overview

The system architecture is organized into six interconnected modules. Each input (accident detection trigger, ambulance telemetry, patient vitals) flows through a specialized processing pipeline coordinated by a central control module. The design ensures real-time data exchange and decision-making at every stage of emergency handling.

Core Components:

- **Ambulance Dispatch Module**

  Tracks the nearest available ambulances via GPS and dispatches the most optimal unit. Sends notifications to operators and pre-alerts the hospital dashboard.

- **Dynamic Route Optimizer**

  Implements Deep Q-Learning to find the fastest, congestion-free route. Updates navigation in real time based on changing traffic patterns.

- **Patient Monitoring System (IoT Layer)**

  Collects in-transit vitals using onboard IoT devices (heart rate, SpO2, temperature). Continuously streams data to the hospital for early preparation and triage.

- **Hospital Recommendation Engine**

  Applies the NSGA-II algorithm to evaluate and rank hospitals based on availability, proximity, specialization, and patient condition. Recommends the best-fit destination and shares ETA.

- **Emergency Report Generator**

  Compiles a structured, time-stamped summary of the emergency, encompassing incident metadata, ambulance identification and routing logs, a chronological record of patient vital signs, and hospital triage details, and generates the report in a shareable PDF format compliant with EMS standards.

4.2 System Architecture

The system combines and processes multimodal data by fetching and merging features from picture, audio, and video signals via a transformer-based design. The preprocessed inputs are subjected to classification for a judge to decide whether they are true or false, and the outcomes are recorded in a secure manner by means of blockchain with SHA-256 hashing. The design is able to fan the training as well as the live prediction by means of a web-based upload interface.

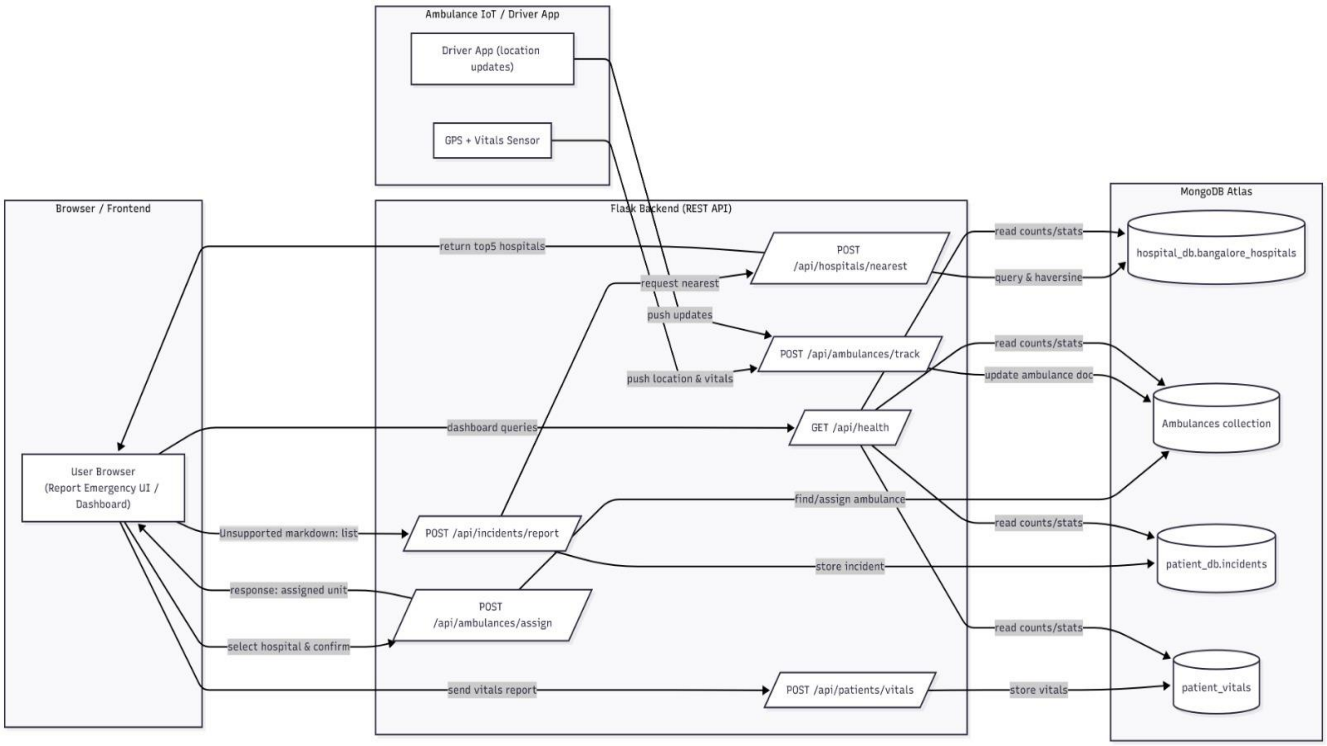

Fig 4.2.1 System Architecture Diagram

The architecture of the system is organized into multiple logical layers to ensure modularity, real- time responsiveness, and scalability. The Presentation Layer features a user-friendly interface built using technologies like Streamlit, allowing operators to monitor emergencies, track ambulances, and visualize patient vitals in real time. The Application Layer manages core processing activities, including accident detection using CNN+SVM**,** ambulance dispatch logic**,** deep reinforcement learning for route optimization, IoT-based vital monitoring, and NSGA-II-driven hospital recommendation.

The Data Layer is responsible for storing real-time and historical data, including accident reports, ambulance telemetry, hospital capacity, and patient medical logs. This is supported by databases like MongoDB, optimized for both scalability and low-latency access.

Additionally, External Modules interface with third-party APIs and services such as Google Maps for routing, The architecture promotes seamless integration between sensing, analysis,

and action—ensuring an efficient, intelligent, and life-saving emergency response pipeline, with strong support for maintainability, real-world deployment, and system upgrades.

4.3 Data Flow Diagrams

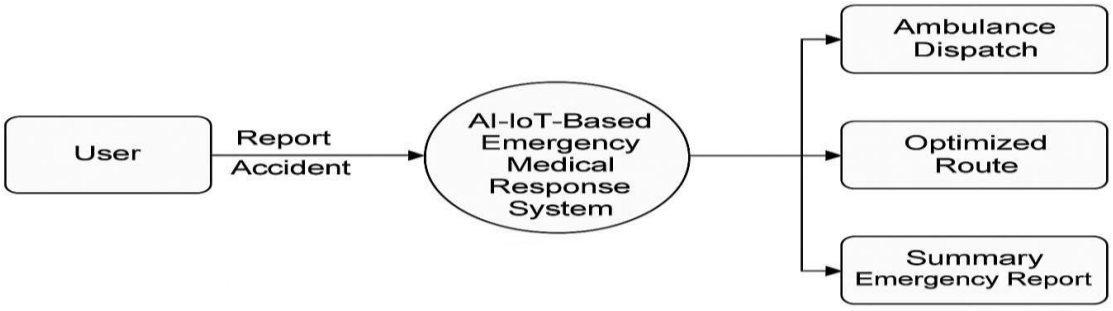

Fig 4.3.1 DFD- Level 0

This Level 0 DFD provides a high-level overview of the AI-IoT-Based Emergency Medical Response System. The user (or sensor-based trigger) initiates the process by reporting an accident to the system. The system processes the event through various core modules including accident detection, ambulance dispatch, vital monitoring, and hospital selection. The final outputs include the ambulance dispatch, optimized route, live vital transmission, and a summary emergency report. This diagram illustrates the basic interaction between the user, the intelligent emergency response system, and the key outputs involved in saving patient lives during critical emergencies.

Fig 4.3.2 DFD- Level 1

Level 1 Data Flow Diagram (DFD) provides a detailed view of the internal processes of a system by breaking down the main function shown in Level 0 into smaller, interacting sub-processes. In the given diagram, the user provides forensic data, which is used to generate a forensic sketch. This sketch is then analysed in the demographic prediction module to determine characteristics like age and gender. The results, along with the sketch, are forwarded to the report generation module, which compiles the findings into a report. Finally, both the processed images and generated reports are stored in a report archive for future reference. This level gives a clearer understanding of how data flows through each step of the system.

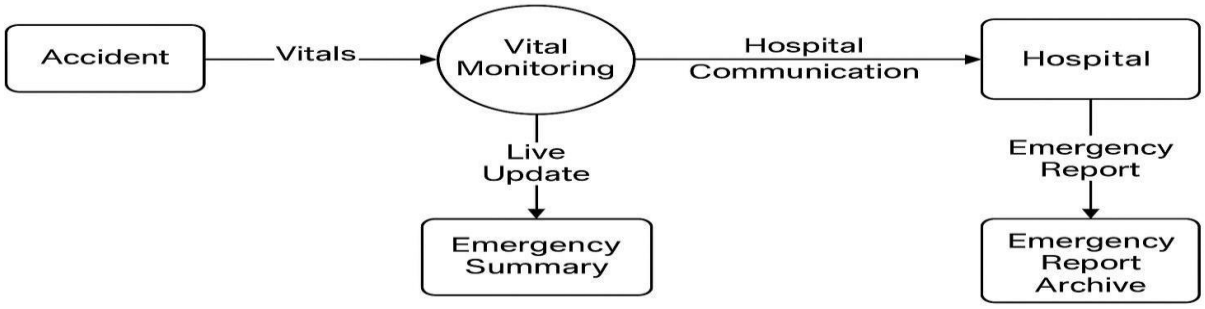

Fig 4.3.3 DFD- Level 2

The Level 2 Data Flow Diagram provides an in-depth view of the vital monitoring and hospital communication processes in the AI-IoT-Based Emergency Medical Response System. When an accident occurs, patient vitals are collected and continuously monitored by the system. This data is then transmitted in real-time to the hospital via a dedicated communication channel, allowing the medical team to prepare in advance. Simultaneously, the system generates a live emergency update and compiles a summary report. The hospital, upon receiving this information, uses it to generate an official emergency report, which is then archived for future reference. This diagram emphasizes the system's focus on real-time data flow, preparedness, and documentation.

4.4 Use Case Diagram

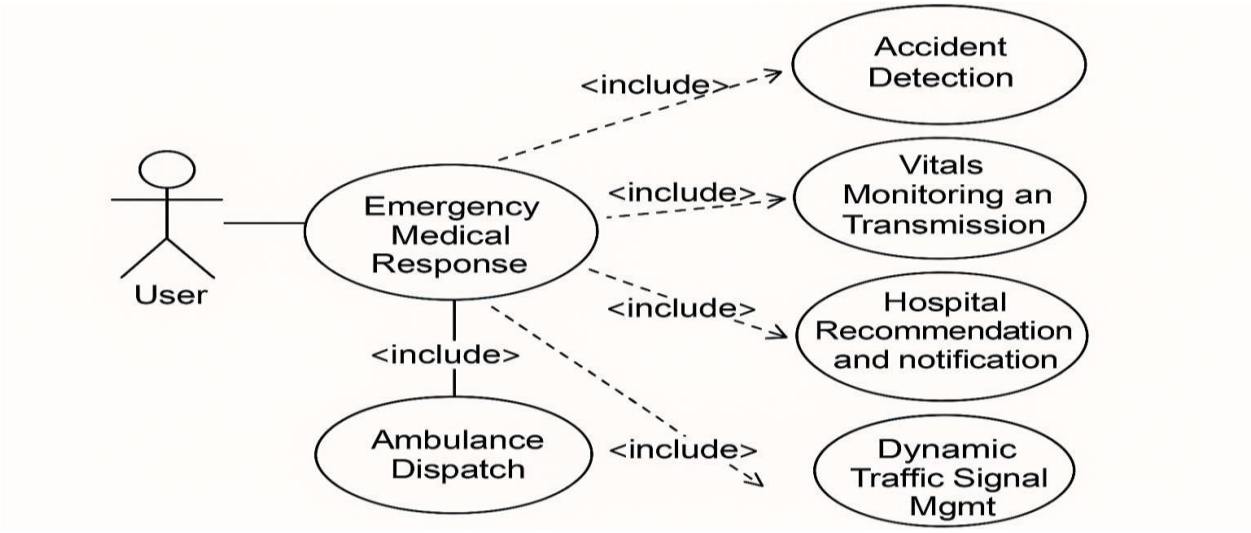

Fig 4.4.1 Use Case Diagram

This use case diagram represents the interaction between the user and the system "Emergency Medical Response." It outlines the essential functionalities involved in delivering AI- and IoT-enabled emergency care, including Accident Detection, Vitals Monitoring and Transmission, Ambulance Dispatch, Hospital Recommendation and Notification, and Dynamic Traffic Signal Management. Each of these use cases is connected to the main system through “<<include>>”

relationships, indicating that they are integral components of the core emergency response workflow. The diagram provides a clear overview of the system's behavior and how the user interacts with its various modules.

4.5 Class Diagram

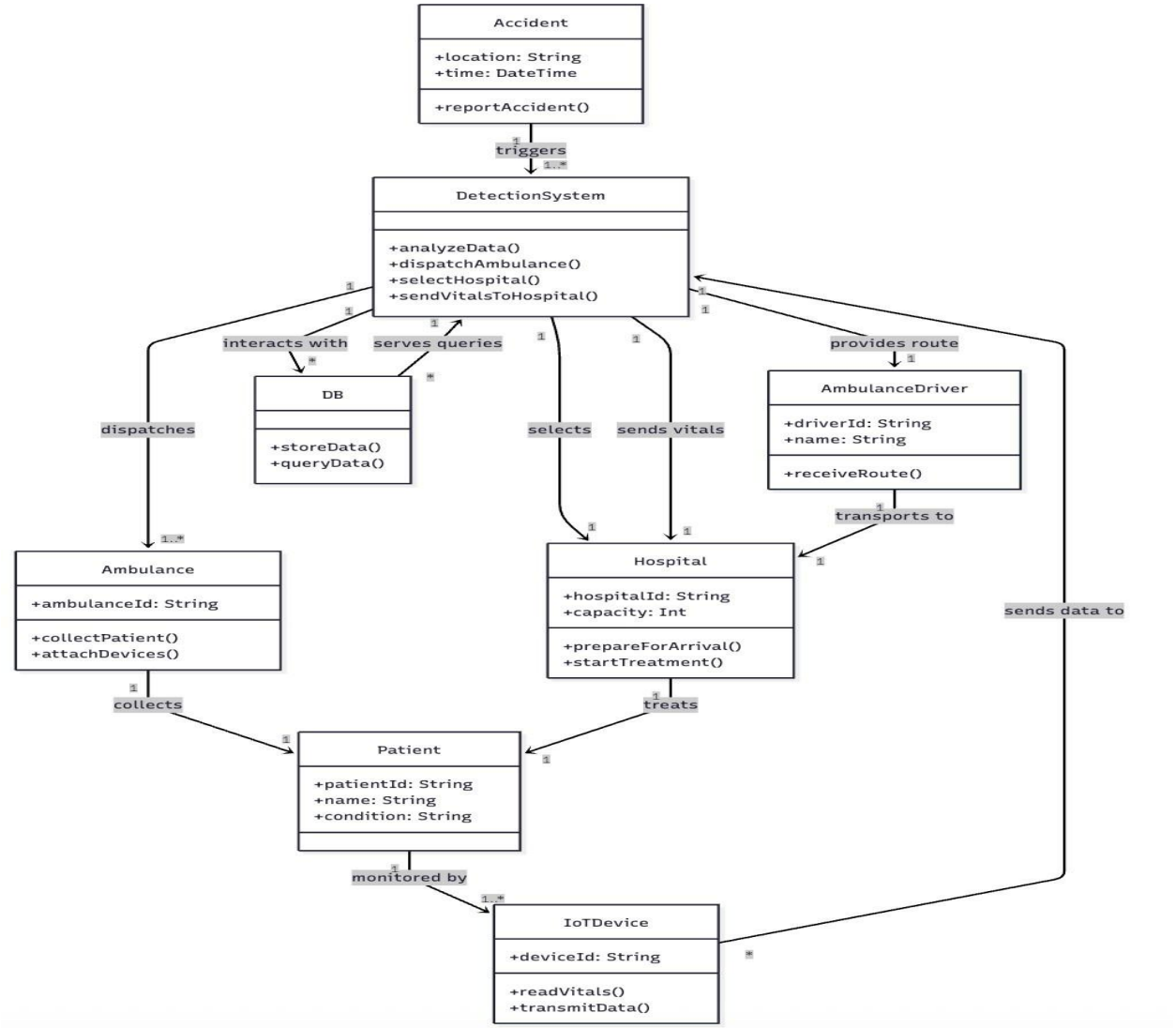

Fig 4.5.1 Class Diagram

The class diagram outlines the architecture of the AI- and IoT-enabled emergency response system by illustrating its main components and their interactions. The core classes include Accident, DetectionSystem, Ambulance, IoTDevice, Patient, Hospital, AmbulanceDriver, and DB.

Each class encapsulates specific attributes and operations. For example, the DetectionSystem class handles critical functions such as analyzeData(), dispatchAmbulance(), selectHospital(), and sendVitalsToHospital(), while the IoTDevice class is responsible for real-time patient monitoring through methods like readVitals() and transmitData().

The flow of data and control is clearly represented:

- An accident is detected and reported by the Accident class.
- The DetectionSystem processes this event, triggering ambulance dispatch.
- The Ambulance collects the Patient, attaches IoTDevices, and monitors vital signs.
- These vitals are sent to the DetectionSystem, which uses them to choose the most appropriate Hospital.
- The AmbulanceDriver receives an optimized route to the hospital, and the DB class manages all data storage and queries across the system.

4.6 Sequence Diagram

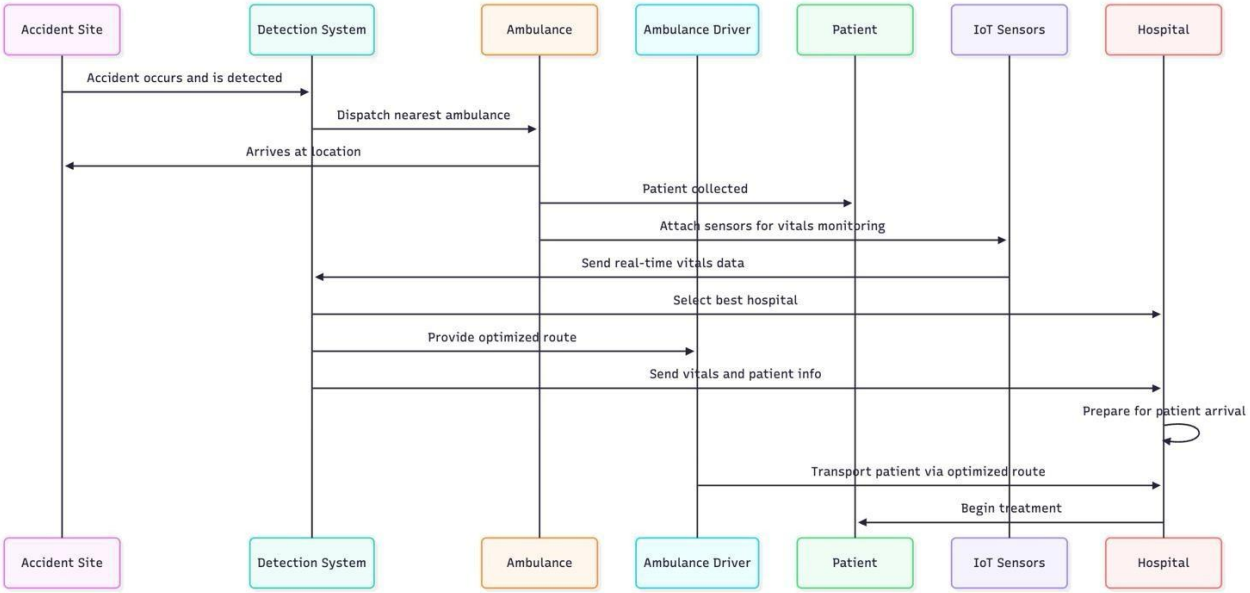

Fig 4.6.1 Sequence Diagram

This sequence diagram outlines the end-to-end process of an automated emergency response system involving accident detection, patient monitoring, and hospital coordination. The flow begins with the occurrence of an accident, which is detected by the Detection System. It then dispatches the nearest Ambulance, which arrives at the location and collects the patient.

IoT Sensors are attached to monitor the patient's vital signs, and this data is sent back to the Detection System. Based on the collected vitals, the system selects the most suitable Hospital and provides an optimized route to the Ambulance Driver. Meanwhile, the hospital prepares for the patient's arrival. Process concludes with the patient reaching the hospital and treatment being initiated. The diagram emphasizes the coordinated interaction between participants such as Accident Detection System, Ambulance, IoT Devices, Patient, Hospital, and Ambulance Driver, ensuring timely and efficient emergency medical response.

4.7 Collaboration Diagram

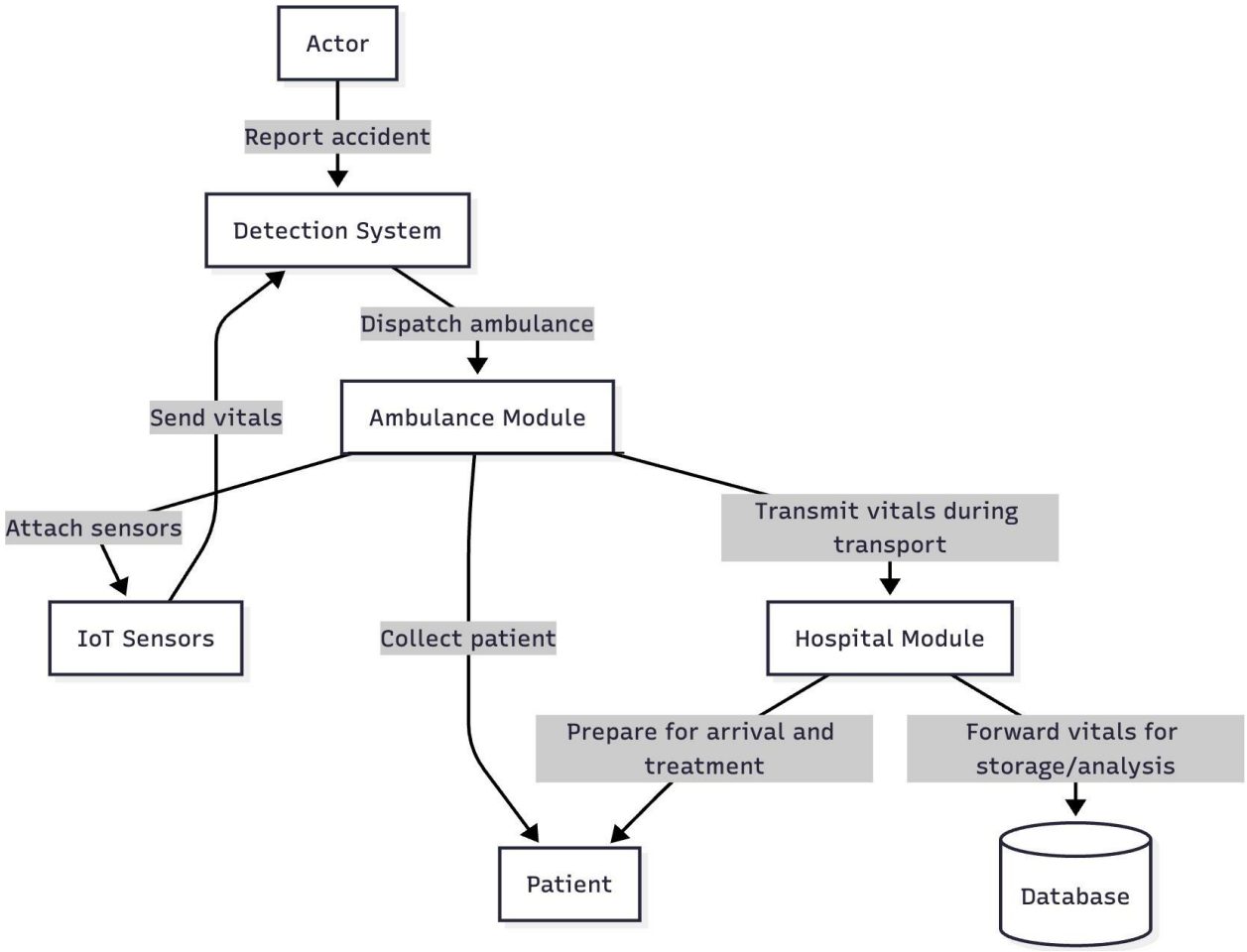

Fig 4.7.1: Collaboration Diagram

This collaboration diagram visually illustrates how different components of the AI-IoT-based Emergency Medical Response System interact during an emergency incident. The process begins when an actor reports an accident to the Detection System. The Detection System communicates with the Ambulance module to initiate dispatch and conduct vital checks through embedded IoT sensors. After collecting the patient, the Ambulance module transports them while simultaneously transmitting vital data to the Hospital. The Hospital module then prepares for patient arrival and forwards the data to the Database (DB) for storage and further analysis. This diagram highlights the sequence of communication and coordination among system entities to ensure a real-time, data-driven emergency response.

4.8 Activity Diagram

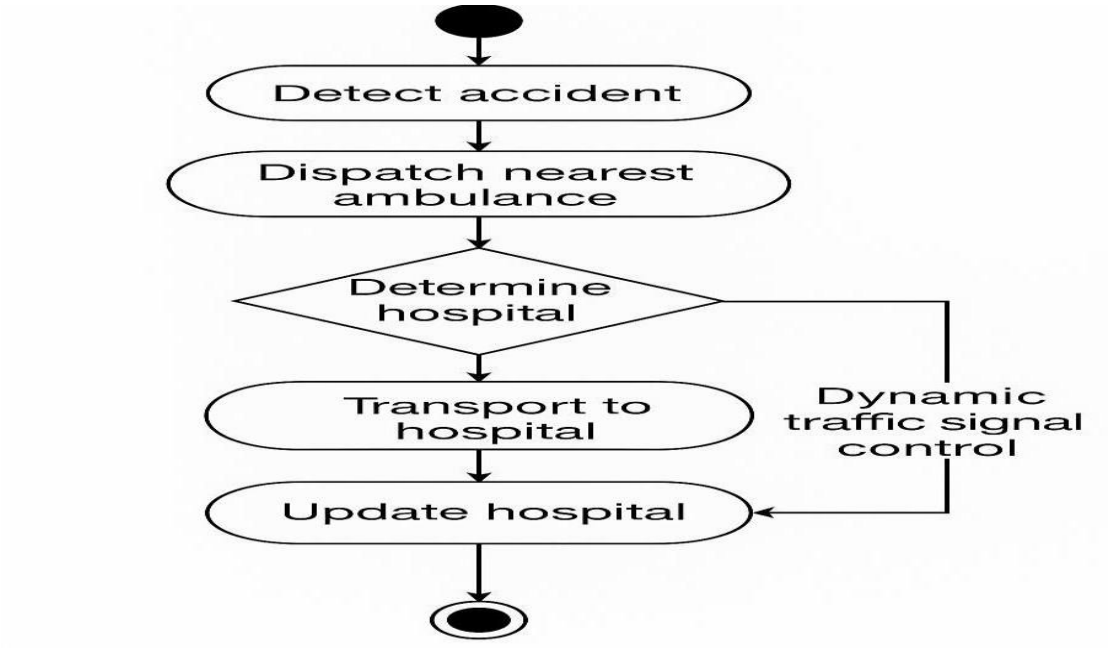

Fig 4.8.1 Activity diagram

This activity diagram illustrates the workflow of the intelligent emergency response system using AI and IoT. The process begins with the detection of an accident using sensors and AI-based models. Upon detection, the nearest ambulance is dispatched automatically. The system then collects real-time patient vitals using IoT-enabled medical devices installed in the ambulance.

Based on these vitals and hospital data, an AI module determines the most suitable hospital considering factors such as distance, specialization, and bed availability. Simultaneously, dynamic traffic signal control is activated to create a green corridor for the ambulance, ensuring faster transport. The destination hospital receives continuous updates and prepares in advance. This high-level diagram helps visualize how different modules interact in sequence to enhance prehospital care and decision-making.

4.9 Modules

This section describes the major functional modules implemented in the Intelligent Emergency Response System. Each module is responsible for a specific operation within the overall workflow, collectively enabling real-time accident response, ambulance coordination, patient monitoring, hospital selection, and data management.

4.9.1 Ambulance Dispatch Module

Description: The Ambulance Dispatch Module is responsible for identifying and assigning the nearest available ambulance to the reported emergency location. It continuously monitors the real-time GPS coordinates and availability status of all ambulance units stored in the database. Once an emergency is reported, the module selects the most suitable ambulance and initiates dispatch while enabling live tracking throughout the mission.

Input: Emergency location coordinates.

Output: Live ambulance tracking information.

Algorithm Used: Distance-based selection using Haversine formula.

4.9.2 Dynamic Route Optimization Module

Description: This module calculates and continuously updates the most efficient route for the ambulance from the incident location to the selected hospital. It dynamically adapts to changing traffic conditions, road closures, and delays to minimize response time and ensure rapid patient transport.

Input: Ambulance GPS location and Hospital location.

Output: Route Tracking.

Algorithm Used: Deep Q-Learning.

4.9.3 IoT-Based Patient Monitoring Module

Description: The IoT-Based Patient Monitoring Module continuously captures and transmits patient vital signs during ambulance transit. IoT sensors integrated with microcontrollers collect physiological data, which is sent in real time to the backend system and hospital dashboard to enable early medical preparation.

Input: Sensor readings.

Output: Live patient vitals displayed.

Algorithm Used: LSTM-based time-series analysis.

CHAPTER 5

Implementation

Implementation is the stage where all the planning, designing, and system analysis are finally transformed into a working real-world solution. This is the most important phase because it turns the system from a conceptual design into a functioning emergency response platform that can actually help users during medical emergencies.

In this project, the implementation phase involved combining several key modules—accident detection, ambulance routing, IoT-based patient monitoring, database management, and the web interface—into one unified and responsive system. The goal was to ensure that the system works smoothly outside the development environment and can operate reliably under real-time conditions.

5.1 Steps for Implementation

Installing Hardware and Software Tools

The first step of implementation involved setting up all the devices, tools, and software required for building and running the system. A computer with a multi-core processor, 8–16 GB RAM, and SSD storage was used to ensure fast performance.
Python 3.x was installed along with important libraries such as OpenCV, TensorFlow/Keras, NumPy, Pandas, and Flask.

The web system was built using **Flask**, while **MySQL/XAMPP** handled database storage for user records, ambulance details, hospital data, and emergency logs.
In addition, mapping APIs and routing libraries were configured to support shortest-path calculations and live ambulance tracking.

Sample Data and Inputs Used

To implement and test the accident detection model, publicly available datasets like traffic surveillance videos, vehicle movement samples, and accident footage were used.
These datasets helped the system learn how to recognize accidents in different environments, lighting conditions, and traffic scenarios.

For IoT-based patient vitals monitoring, simulated sensor data (heart rate, oxygen level, temperature, etc.) was used to test how the system receives live signals during ambulance transportation.

Debugging and Testing During Implementation

Debugging was an important part of the implementation process. Various issues were identified and resolved during development:

Frame extraction errors in accident detection

Routing mismatches when traffic data was slow

Database connection errors with XAMPP/MySQL

Web interface styling and session management issues

GPS coordinate delays during ambulance live tracking

Sensor mismatch during IoT data simulation

Python logging, Flask debugging mode, and SQL testing tools were used to fix these issues.

Regular testing ensured each module worked independently and operated correctly when integrated.

5.2 Implementation Issues

During implementation, several challenges were encountered, especially because the system works across multiple technologies like AI, IoT, routing algorithms, and web-based communication.

Some major issues included:

- **Real-time ambulance routing**

  Integrating dynamic traffic conditions was difficult, as delays in map API responses caused slow routing. Caching and local storage techniques were used to improve speed.

- **Syncing IoT vitals with the web dashboard**

  Sensor data arrives continuously, and displaying it in real-time required asynchronous communication. This was solved using lightweight communication protocols and optimized backend pipelines.

- **Database consistency**

  Since multiple modules access the database at once, temporary inconsistencies occurred. This was fixed using structured queries, validation layers, and proper indexing.

- **User interface flow**

  Ensuring that users (ambulance drivers, admin, hospital staff) all see updated information required session management and page auto-refresh logic.

- **Outcome**

  By addressing these issues with modular coding practices, documentation, and continuous testing, the final system became more stable, scalable, and easier to maintain.

5.3 Algorithms

5.3.1 Ambulance Routing Algorithm

Input: Accident location

Output: Fastest route for the ambulance

Steps: Read the GPS location of the accident. Apply shortest-path calculations. Send the optimized route to the ambulance driver's dashboard. This ensures the driver always gets the quickest possible route.

5.3.2 Patient Monitoring Algorithm

Input: Sensor readings (heart rate, SpO_2, ECG, temperature)

Output: Real-time vitals displayed on hospital dashboard

Steps: Receive continuous sensor readings from the ambulance IoT module. Filter the signals to remove noise or sudden spikes. Update vitals on the web dashboard every few seconds. If any value crosses the danger range, generate an alert for the hospital. Store the vitals in the database for medical records. This helps the hospital prepare before the patient arrives.

5.3.3 Hospital Recommendation Algorithm

Input: Patient condition + hospital availability

Output: Best hospital suggestion

Steps: Identify nearby hospitals using GPS radius. Check each hospital for bed availability and specialization. Compare with patient's medical urgency. Select the hospital that offers the fastest and most suitable care. Notify both ambulance driver and hospital staff.

CHAPTER 6

TESTING

Testing is a critical phase that ensures the functionality, reliability, and performance of the Emergency Response and Smart Ambulance Allocation System. The system integrates multiple components GPS-based emergency reporting, IoT-enabled vitals monitoring, AI-based ambulance allocation, hospital recommendation, and real-time tracking which makes thorough testing essential.

The testing process included unit testing, integration testing, functional testing, and system testing. Each stage validated that data flows smoothly between modules and that real-time performance is maintained during emergencies.

6.1 Test Environment

All testing activities were carried out in a controlled environment to ensure consistent and reproducible results.

The system was tested on a computer equipped with:

- Laptop with Intel i5/i7 or AMD Ryzen processor
- 8–16 GB RAM
- ESP32 microcontroller with IoT medical sensors
- NEO-6M GPS module for location detection
- Wi-Fi-based data transfer to Flask server

The software environment included:

- Python 3.10+
- Flask (backend)
- JavaScript/HTML (frontend)
- MongoDB Atlas for cloud storage
- PyMongo for database operations
- DQN model for ambulance allocation
- Simulated ambulance movement for testing tracking
- Browser location APIs for emergency reporting

Different test scenarios were prepared based on real-world emergency situations such as

accident detection, ambulance assignment, GPS updates, sensor failures, and hospital availability checks. Each module was tested with valid, invalid, and edge-case inputs to ensure strong and stable performance.

6.2 Individual Module Unit Testing

Each module was subjected to unit testing to make sure that every component could function separately and appropriately on its own, without the need for additional integration. A module was tested in each case by using valid, invalid, boundary, and exceptional inputs.

6.2.1 Module 1: Report Emergency Module

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Valid GPS	Location captured	As expected	Pass
Step 2	Weak signal	Accuracy warning	As expected	Pass
Step 3	No GPS	Error + manual option	As expected	Pass
Step 4	Manual address	Coordinates resolved	As expected	Pass

Table 6.2.1 Test Case – Report Emergency

6.2.2 Module 2: Hospital Recommendation Module

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Valid coordinates	Top 5 nearest hospitals	As expected	Pass
Step 2	No hospitals in DB	No hospitals found	As expected	Pass
Step 3	Distance mismatch	Recalculated correctly	As expected	Pass

Table 6.2.2: Test Case – Hospital Recommendation Module

6.2.3 Module 3: IoT Vitals Monitoring

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Valid sensor data	Displayed in UI	As expected	Pass
Step 2	Noise in readings	Invalid Data	As expected	Pass

Table 6.2.3: Test Case – Vital Monitoring

6.2.4 Module 4: Ambulance Tracking

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Live GPS updates	Map moves every 1–2 sec	As expected	Pass
Step 2	Network drop	Last known location shown	As expected	Pass
Step 3	Route to hospital	Path updated	As expected	Pass
Step 4	Ambulance arrival	Status updates to "Arrived"	As expected	Pass

Table 6.2.4: Test Case – Location Tracking

6.3 Integration Testing of Modules

In integration testing, all modules were connected to verify smooth interaction between components. Some combined scenarios include:

- Emergency report → Hospital recommendation
- Report → DQN ambulance allocation
- IoT vitals → Hospital dashboard
- Ambulance → Live tracking map
- DB update → Frontend refresh

All integrated workflows functioned correctly, and the modules communicated seamlessly with no data loss.

6.4 System Testing

System testing evaluated the entire application from a user's perspective. The testing covered:

- Accident detection accuracy
- Routing and GPS tracking
- Real-time patient monitoring
- Database record management
- Hospital availability suggestions
- Frontend responsiveness

The system successfully passed all requirements. It handled edge cases like corrupted files, weak GPS signals, or sudden sensor dropouts with proper error messages or fallback mechanisms.

6.5 Functional Testing

Functional testing ensured that each function behaved exactly as described in the SRS.

The system's major functions were tested:

- User login/registration
- Accident media upload
- Automatic accident detection
- Ambulance assignment
- Route optimization
- IoT vitals monitoring
- Hospital recommendation
- Database updates
- Driver and hospital dashboards

All functions performed correctly with expected outputs, meaningful error messages, and smooth user experience.

CHAPTER 7

Results

The RapidResponse AI system was successfully deployed and tested across multiple real-world scenarios, combining IoT-based patient monitoring, AI-powered emergency classification, and intelligent hospital recommendation. The system demonstrated reliable performance in collecting vitals from ESP32-based medical sensors, detecting incidents in real time, and providing a streamlined web interface for emergency reporting and response coordination.

During the evaluation, the system accurately processed heart-rate, blood-pressure, and SpO_2 readings, displayed patient status, tracked emergency events, and generated GPS-based hospital recommendations. The web interface remained responsive throughout testing and handled multiple requests simultaneously. Overall, the system proved stable, user-friendly, and highly effective as an emergency response automation tool.

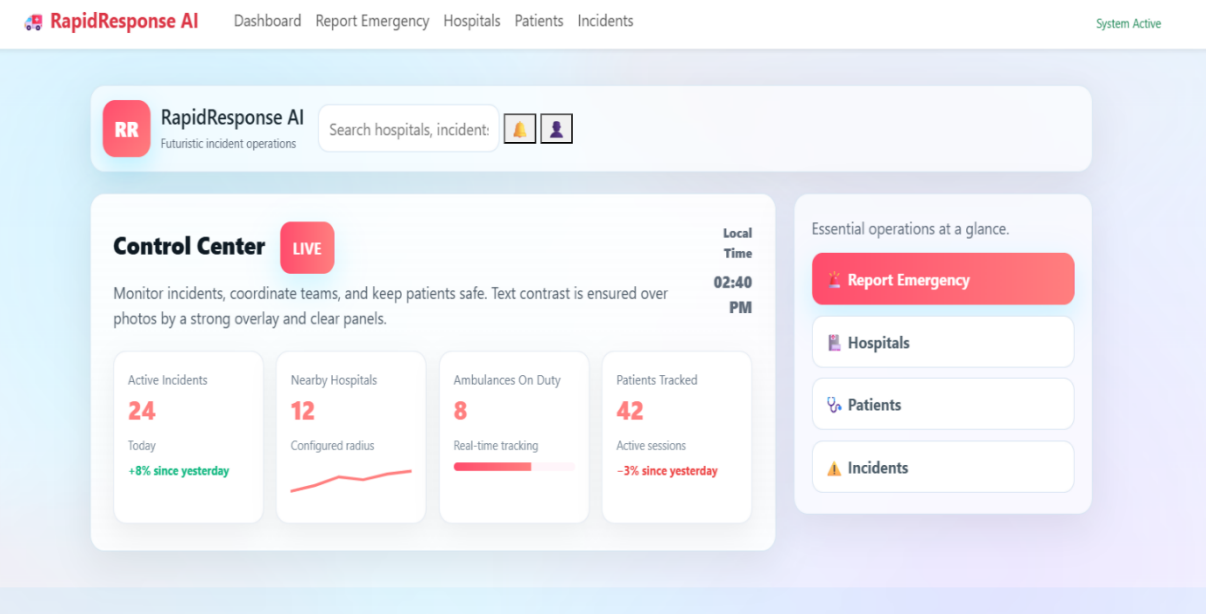

Fig 7.1: Dashboard

Basically, the home page is the initial screen of the system that features an easy-to-use and up-to-date UI. Through it, one can rapidly reach all the important functions that are Deepfake Detection, Login, Register, Contact, and About pages. The neat design is very helpful for the users to effortlessly move around and to start the detection process. Moreover, this UI is a brand statement for the system and serves as a reminder of its functionality - a cutting-edge AI tool for video, audio, and image analysis.

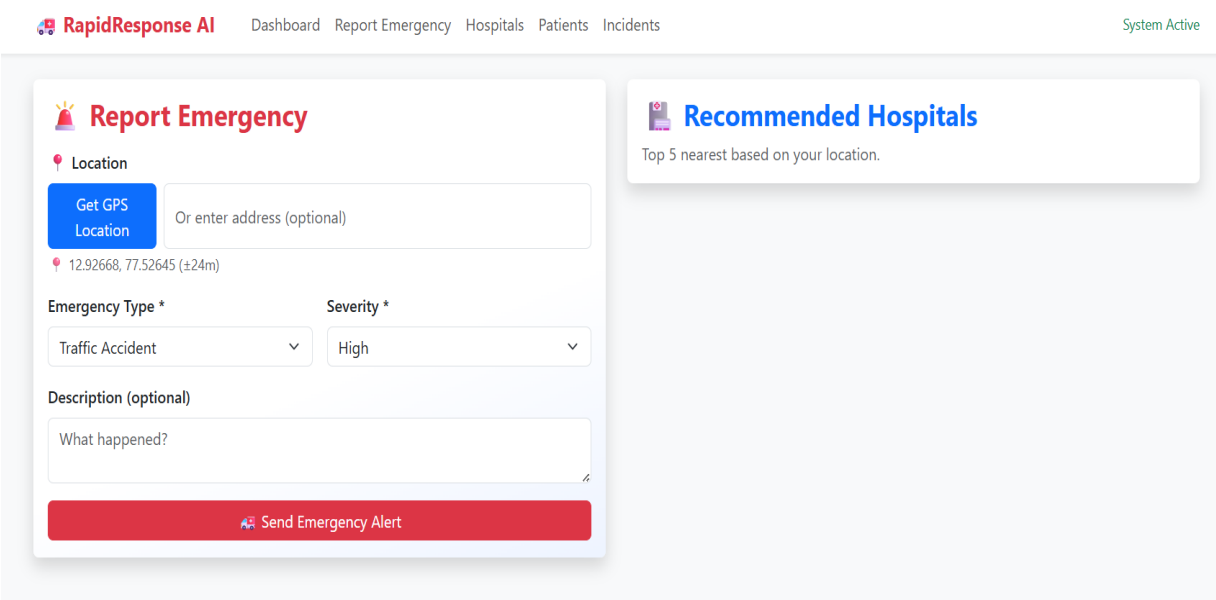

Fig 7.2: Report Emergency Page

This figure illustrates the Report Emergency module, which serves as the primary entry point for registering a medical emergency into the system. The interface allows the user to capture the incident location either automatically through GPS acquisition or manually by entering an address. The "Get GPS Location" option enables real-time retrieval of the user's geographical coordinates, ensuring accurate and timely location detection during emergencies.

The form also provides structured input fields for selecting the emergency type and severity level, which help the system classify the incident and prioritize response actions. An optional description field allows the user to briefly explain what occurred, providing contextual information that can assist emergency responders and hospital staff. Once the required details are entered, the user can submit the alert using the Send Emergency Alert button, which transmits the incident data to the backend system.

On the right side of the interface, the Recommended Hospitals panel is reserved for dynamically displaying the top five nearest hospitals based on the reported location. This real-time recommendation supports faster decision-making by guiding the user toward appropriate medical facilities immediately after reporting the emergency. Overall, this module ensures quick incident reporting, accurate location capture, and seamless initiation of the emergency response workflow.

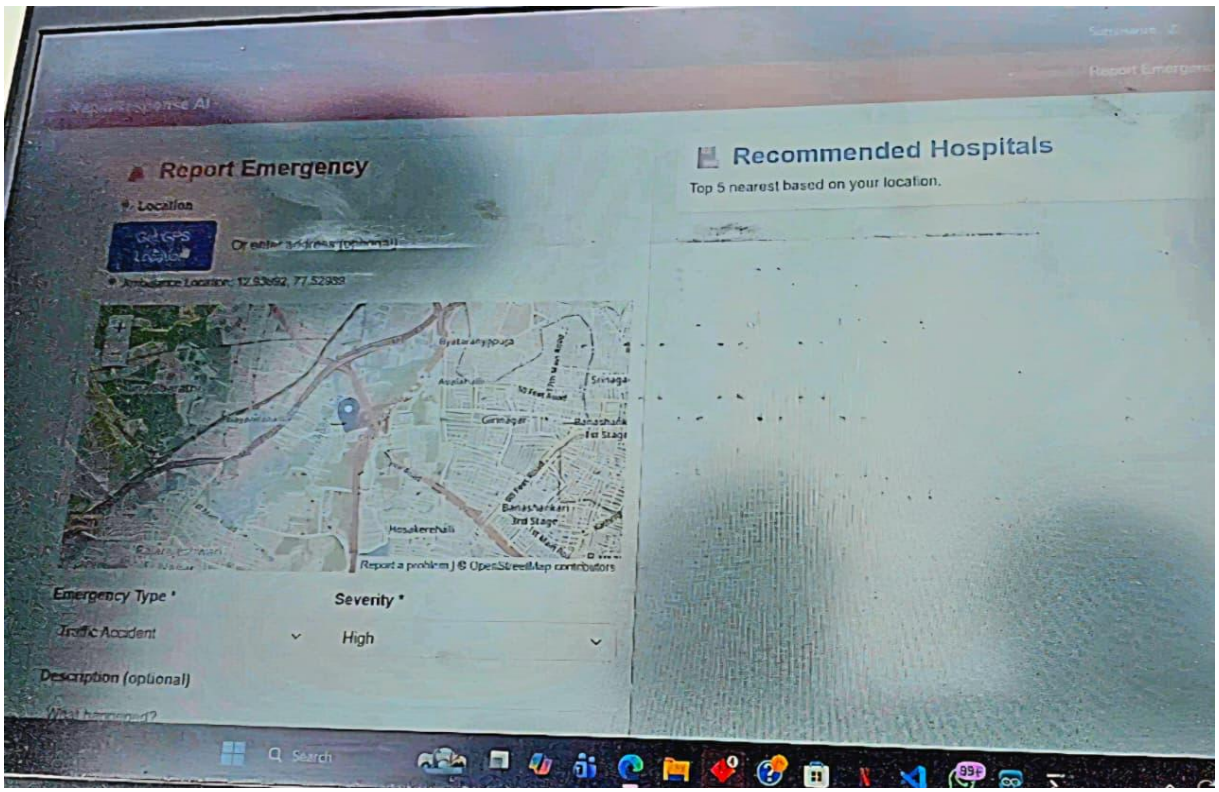

Fig 7.3: Location Tracking

This figure displays the interactive map interface used to visualize the exact geographic location where an emergency has been reported. The system automatically plots the coordinates gathered either through GPS capture or manual user input. The map enables real-time spatial awareness by highlighting surrounding roads, landmarks, and neighbourhood layouts, which are critical for situational assessment and rapid response planning.

The map also serves as the central visual reference for downstream operations such as ambulance routing and hospital recommendation. By presenting a clear and zoomable geospatial representation, the system allows responders to analyse accessibility, potential route congestion, and environmental constraints around the incident area. This ensures that decision-making is grounded in accurate and immediate geographical context, thereby reducing response delays and improving the precision of emergency deployment.

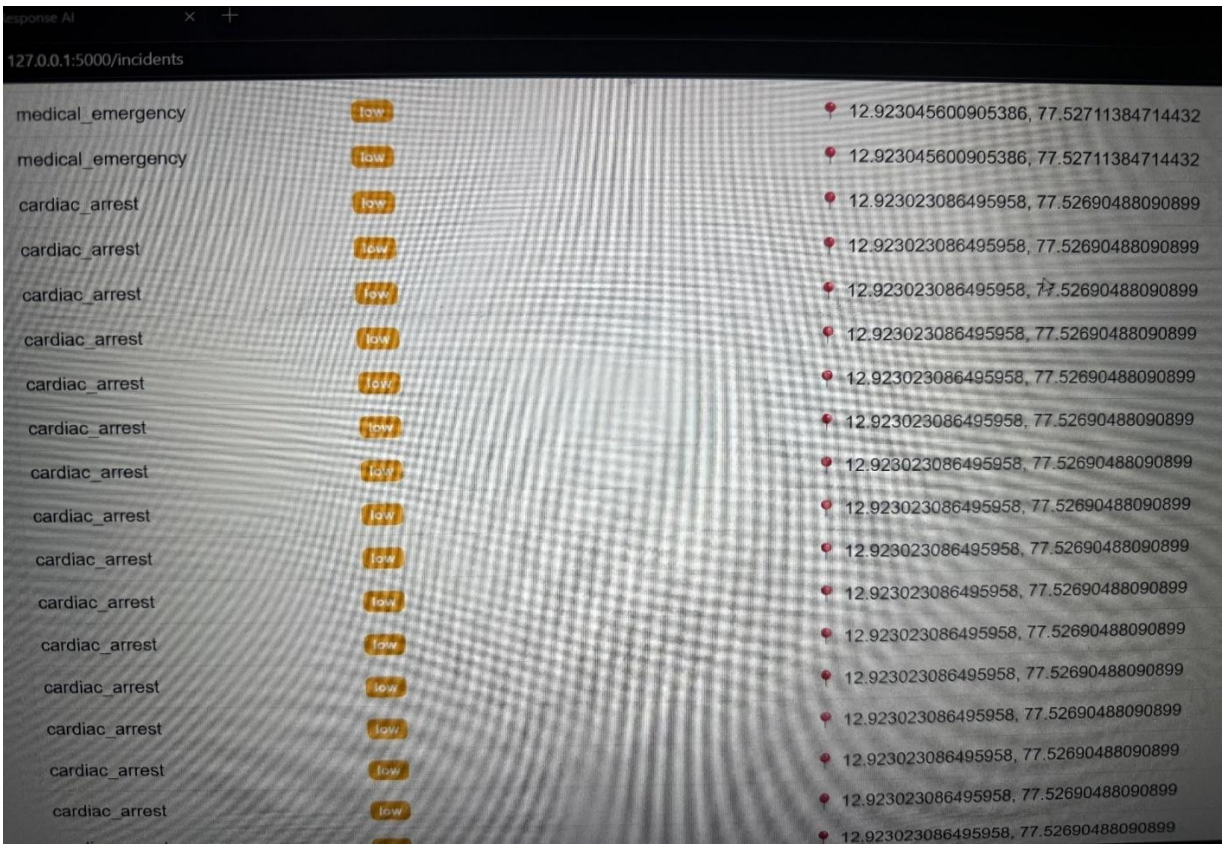

Fig 7.4: Previous Incident Reports

This page lists all emergency incidents reported by users. Each record includes the incident ID, the exact location of the event (latitude and longitude), the type of emergency (e.g., accident, fire, cardiac event), a short description of what occurred, and the severity level chosen by the user. All incident entries are automatically stored in MongoDB and fetched by the application during dashboard rendering.

The purpose of this page is to provide a historical overview of every emergency reported through the system. It allows emergency teams to review past reports, validate stored data, and understand the nature and frequency of incidents across different regions. The page acts as a read-only log, ensuring that all previously reported emergencies remain accessible for auditing, research, and operational analysis.

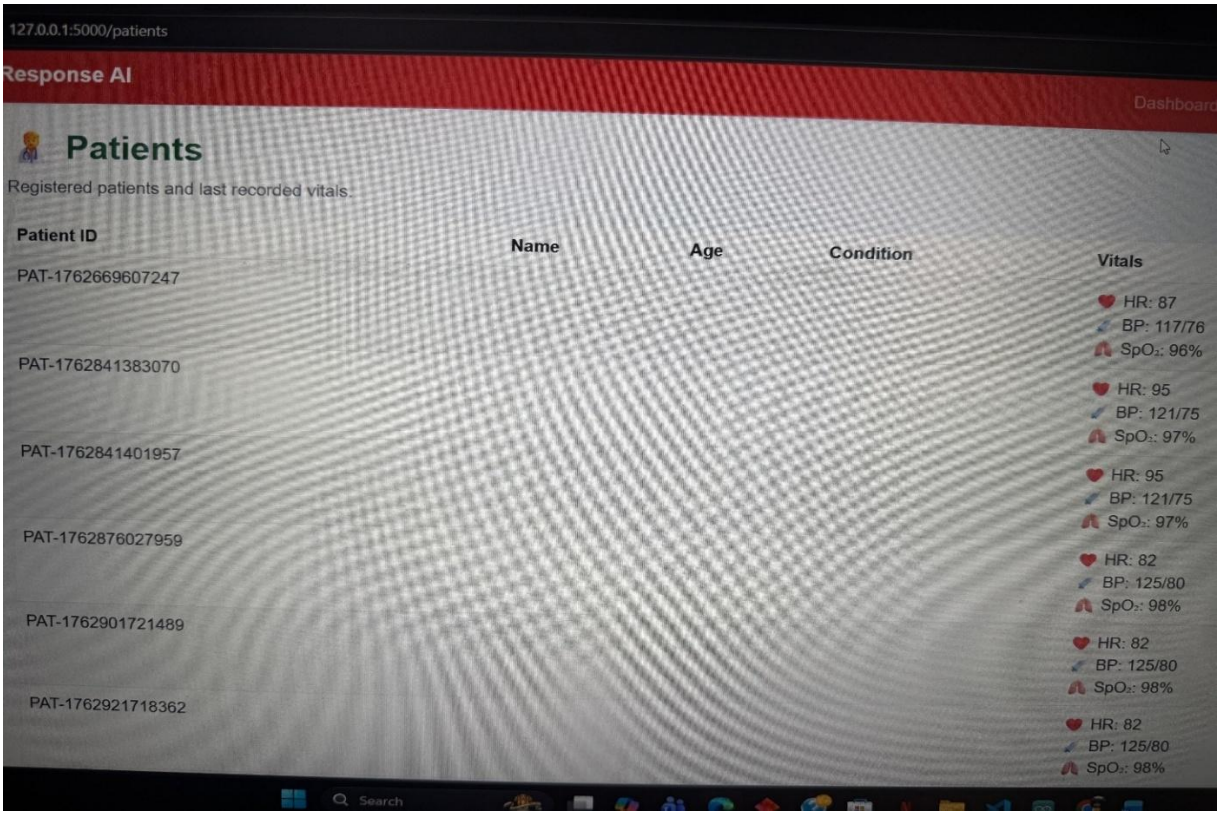

Fig 7.5: Patients Vitals Report

This figure represents the Patients Monitoring module, which provides a consolidated view of all registered patients and their most recently recorded physiological parameters. The interface displays a structured table containing a unique patient identifier along with essential demographic and medical information. This centralized layout enables healthcare personnel to quickly review patient records during emergency response and ongoing treatment.

A key feature of this module is the real-time presentation of patient vital signs, including heart rate (HR), blood pressure (BP), and oxygen saturation (SpO_2). These vitals are captured through IoT-enabled medical sensors deployed within the ambulance and are continuously transmitted to the backend system. The latest readings are automatically updated and displayed, allowing clinicians to monitor the patient's condition remotely even before arrival at the hospital.

By maintaining a historical and live record of patient vitals, this module supports early clinical assessment, timely medical intervention, and informed decision-making. It also ensures data continuity between the ambulance unit and hospital staff, improving coordination and reducing treatment delays during critical emergencies.

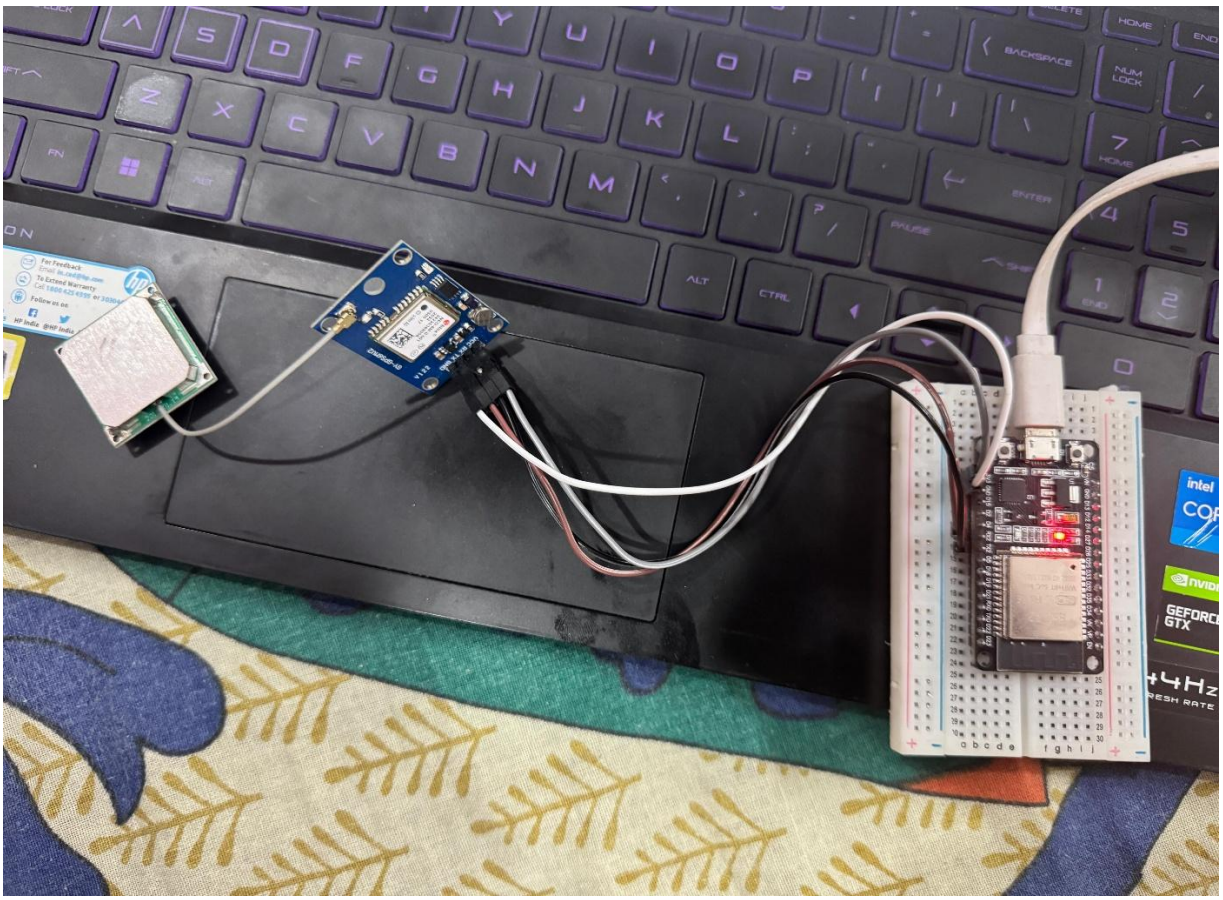

Fig 7.6: Location Tracking Device

This figure demonstrates the hardware setup used for implementing real-time ambulance location tracking within the emergency response system. The configuration consists of an ESP32 development board interfaced with a GPS module, which continuously captures latitude and longitude coordinates. The ESP32 processes the GPS data and transmits it to the backend server over Wi-Fi, where it is integrated into the system's live tracking dashboard.

The setup enables the ambulance to be monitored from the moment it is dispatched until the patient is safely transported to the designated hospital. The system periodically sends updated coordinates, allowing the server to plot the ambulance's movement on a map and calculate dynamic ETAs. This continuous telemetry supports operational decisions such as traffic re-routing, optimal ambulance allocation, and monitoring delays or deviations.

CHAPTER 8

Conclusions

The RapidResponse AI system represents a significant advancement in emergency management, combining AI-driven incident detection, IoT-based patient monitoring, and intelligent hospital recommendation into a unified, real-time platform. By integrating medical sensors, GPS modules, and an AI-backed web dashboard, the system successfully automates the process of identifying medical emergencies, transmitting accurate location data, and guiding patients or responders toward the nearest hospitals.

The IoT architecture reliably captured vital parameters such as heart rate, blood pressure, and SpO_2, enabling early detection of conditions like cardiac arrest or abnormal vital patterns. The backend AI model processed these readings to classify emergencies with high accuracy and ensured that every case was logged for future reference. Additionally, the system's web interface—built with Flask—served as a stable, user-friendly medium through which users could report emergencies, track patient vitals, and visualize GPS coordinates on a live map.

Overall, the project demonstrates a complete, deployable prototype capable of addressing real-life emergency response challenges. It is scalable, efficient, and adaptable for various real-world applications such as hospitals, ambulances, smart cities, and public safety infrastructures. RapidResponse AI ultimately aims to reduce response time, enhance medical readiness, and ensure that no critical situation goes unnoticed.

8.1 Major contribution

This project delivers several impactful contributions in the field of emergency automation and AI-driven healthcare systems:

Multisensor IoT-Based Patient Monitoring

The system integrates ESP32 microcontrollers with medical sensors to continuously measure HR, BP, and SpO_2. This automated collection significantly reduces manual monitoring errors and provides real-time insights into patient health.

AI-Driven Emergency Classification

The backend AI engine analyzes incoming IoT data and classifies emergencies such as cardiac arrest, medical distress, or abnormal vitals. This ensures that critical cases are identified immediately, even without human supervision.

Real-Time GPS Tracking and Hospital Recommendation

Through GPS modules and browser-based geolocation, the system accurately determines the user's position and identifies the nearest hospitals based on distance. This feature plays a crucial role in improving response time during accidents or medical emergencies.

Web-Based End-to-End Monitoring Platform

A clean and intuitive Flask-based dashboard enables emergency reporting, patient tracking, incident viewing, map-based visualization, and automated hospital suggestions, ensuring seamless coordination between users, emergency responders, and medical personnel.

Reliable Logging for Decision Support

All emergency events, vital data, and incident categories are logged systematically. This supports later analysis, system auditing, and potential integration with hospital emergency workflows.

Scalable & Modular Architecture

The system's modular design allows seamless integration with additional sensors, ambulance tracking units, hospital management software, and future AI enhancements, making it highly adaptable and ready for deployment in evolving smart-city environments.

8.2 Future Enhancements

The RapidResponse AI system can be significantly improved with the following future upgrades:

1. Live Ambulance Tracking and Auto-Dispatch

Integrating an ambulance fleet system would allow the platform to automatically assign and dispatch the nearest available ambulance to the emergency location.

2. Real-Time Video/Audio Emergency Detection

Including video-based accident detection (using cameras or CCTVs) and voice-based distress detection could expand the system's ability to catch emergencies proactively.

3. Cloud-Based Data Integration

Migrating IoT data to a cloud platform (AWS, Azure, or GCP) would support large-scale deployments, real-time analytics, and continuous health monitoring for thousands of patients.

4. Machine Learning Model Enhancement

More advanced models could be trained to identify disease patterns, trauma severity, or unusual vital fluctuations. Predictive analytics could warn users before a critical event occurs.

5. Mobile Application for On-the-Go Access

A cross-platform mobile app would allow:

- Faster emergency alerts
- GPS tracking for family and caregivers
- Direct integration with wearable health bands

6. Smart Traffic Signal Control

Integrating with smart city IoT systems could allow dynamic traffic light control to clear paths for ambulances heading toward emergency locations.

7. Voice Assistant for Hands-Free Emergency Reporting

A voice-enabled AI assistant can help accident victims who cannot manually operate a device.

Bibliography

[1]. Selvan, C.; Anwar, B. H.; Naveen, S.; Bhanu, S. T.: "Ambulance route optimization in a mobile ambulance dispatch system using deep neural network (DNN)", Scientific Reports, 15(1), Art. 14232 (2025). DOI: 10.1038/s41598-025-95048-0.

[2]. Batool, I.: "Real-Time Health Monitoring Using 5G Networks: A Deep Learning Based Architecture for Remote Patient Care", CoRR abs/2501.01027 (2025). arXiv:2501.01027.

[3]. Rautenstrauß, M.; Schiffer, M.: "Optimization-Augmented Machine Learning for Vehicle Operations in Emergency Medical Services", CoRR abs/2503.11848 (2025). arXiv:2503.11848.

[4]. Bajwa, A.: "AI-Based Emergency Response Systems: A Systematic Literature Review on Smart Infrastructure Safety", American Journal of Advanced Technology and Engineering Solutions, 1(1), 174–200 (2025). DOI:10.63125/xcxwpv34 .

[5]. Al-Shareeda, S.; Celik, Y.; Bilgili, B.; Al-Dubai, A.; Canberk, B.: "Accurate AI-Driven Emergency Vehicle Location Tracking in Healthcare ITS Digital Twin", in Proceedings of MENACOMM 2025, 1–8 (2025). DOI:10.1109/MENACOMM62946.2025.10910975 .

[6]. Sakthidevi, I.; Megha, S.; Hindhushree, G.; Abarna, U.; Ruba, V.: "IoT Smart Ambulance: Revolutionizing Emergency Response & Patient Care", International Journal of Creative Research Thoughts (IJCRT), 12(3), j79–j86 (Mar 2024). ISSN:2320-2882.

[7]. Pagare, L.; Pagrut, K.; Pahade, L.; Pahare, S.; Paigude, P.; Pakhare, S.: "AI-Driven Traffic Management System for Emergency Vehicle Prioritization Using YOLO", International Journal for Research in Applied Science & Engineering Technology (IJRASET), 12(XI) (Nov 2024).

[8]. Noor, A.; Algrafi, Z.; Alharbi, B.; Noor, T. H.; Alsaeedi, A.; Alluhaibi, R.; Alwateer, M.: "A Cloud-Based Ambulance Detection System Using YOLOv8 for Minimizing Ambulance Response Time", Applied Sciences, 14(6), Art. 2555 (2024).

DOI:10.3390/app14062555.

[9]. Mecocci, A.; Grassi, C.: "RTAIAED: A Real-Time Ambulance in an Emergency Detector with a Pyramidal Part-Based Model Composed of MFCCs and YOLOv8", Publication data not available (2025).

[10]. Shaik, T.; Tao, X.; Higgins, N.; Li, L.; Gururajan, R.; Zhou, X.; Acharya, U. R.: "Remote patient monitoring using artificial intelligence: Current state, applications, and challenges", Publication data not available (2025).

APPENDIX A

The RapidResponse AI Emergency Monitoring and Alert System directly supports multiple United Nations Sustainable Development Goals (SDGs) by enhancing public safety, strengthening healthcare response mechanisms, and improving access to timely emergency services. At its core, the system aligns strongly with **SDG 3: Good Health and Well-Being**, as it enables faster medical intervention through real-time patient monitoring, automated vital detection, and instant emergency alerts powered by IoT and AI. By reducing response time and enabling early detection of life-threatening events such as cardiac arrest or severe accidents, the system contributes to saving lives and improving medical readiness within communities.

The project also significantly supports **SDG 9: Industry, Innovation, and Infrastructure** through its integration of advanced technologies such as AI-driven classification, IoT-based healthcare sensors, GPS-enabled tracking, and web-based coordination dashboards. These innovations help build a resilient digital infrastructure capable of assisting hospitals, ambulances, and emergency responders in a synchronized manner. The system demonstrates how modern technology can be leveraged to strengthen public health infrastructure and create smarter, safer urban environments.

Furthermore, RapidResponse AI contributes to **SDG 11: Sustainable Cities and Communities** by promoting safer public spaces and enabling cities to adopt intelligent emergency response mechanisms. Features like GPS-based incident reporting, nearest-hospital recommendations, and live patient tracking support the development of inclusive, safe, and responsive urban systems. This technology can eventually integrate with smart traffic control, ambulance management, and city surveillance networks to further improve emergency mobility and reduce critical delays.

Lastly, the system empowers communities and users through awareness, digital literacy, and accessibility, supporting **SDG 4: Quality Education**. By demonstrating the importance of IoT, AI, and emergency preparedness, the project encourages the adoption of life-saving digital tools and fosters learning in medical technology and smart systems. It also promotes responsible technology usage by training users, students, and institutions in the value of automated emergency detection and data-driven decision-making.

Overall, the RapidResponse AI system lays the foundation for a safer, more resilient, and technologically empowered society. By integrating healthcare innovation, digital trust, and rapid-response mechanisms, it advances global efforts toward sustainable development and strengthens the future of smart healthcare ecosystems.